CSCI3170
Introduction to Database Systems

10-transaction management

Slides: Michael Yu

Concept of a Transaction

- A ***unit* of program execution** that accesses (and possibly updates) various data items. See *Fund Transfer* example below.

- Transaction: $50 from acct A to acct B
  1. **read**(*A*)
  2. *A* := *A* – 50
  3. **write**(*A*)
  4. **read**(*B*)
  5. *B* := *B* + 50
  6. **write**(*B*)

- A **transaction** will require the usage of multiple **database access operation.** Common operations relevant to transaction processing are (1) Read, (2) Write, and (3) Computation

Fund Transfer (1) Example

- Suppose if transaction fails after step 3 and before step 6, money will be "lost".
  - Leads to inconsistent DB state

- (Property of **Atomicity**):
  - Any database system should ensure that updates of a partially executed transaction are not reflected.
  - i.e., all-or-nothing

E.g., Transaction to transfer $50 from acct A to acct B

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

Fund Transfer (2) Example

- **Durability**
  - Once user has been notified that the transaction has completed (e.g., transfer of the $50 has taken place), it is expected that any updates to database by this transaction must persist even if there are software or hardware failures.

E.g., Transaction to transfer $50 from acct A to acct B

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

Fund Transfer (3) Example

- **Consistency**
  - Implicit integrity constraints
    - E.g., sum of A and B is unchanged by execution of transaction
  - A transaction must always start with a consistent database and leave it in a consistent state upon completion.
  - If a transaction contains a bug or logical error, it can violate constraints and leave the database in a permanently inconsistent state.
  - During the middle of a transaction's execution (e.g., after deducting from account A but before adding to account B, the database may be temporarily in an invalid or inconsistent state.

E.g., Transaction to transfer $50 from acct A to acct B

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

Fund Transfer (4) Example

- **Isolation**
  - ensures that concurrent transactions execute independently without interfering with each other, preventing them from seeing intermediate, inconsistent database states.
  - Suppose between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (A + B less than it should be).

T1	T2
1. **read**(*A*)	
2. *A* := *A* – 50	
3. **write**(*A*)	
	read(A), read(B), print(A+B)
4. **read**(*B*)	
5. *B* := *B* + 50	
6. **write**(*B*)	

Discussion

- Key difficulty for database management system when managing transactions are:
  - Multiple users/transactions might want to read/change same/different data
  - Allowing multiple transactions to update data concurrently can result in complications with consistency of data
  - When several transactions run concurrently, isolation property may be violated

- Concurrency-control scheme is to control interaction among concurrent transactions to prevent them from destroying consistency of database

Summary

- To preserve integrity of data, system must ensure the ACID property
  - **C**onsistency:
    - Every transaction sees a consistent database.
  - **A**tomicity:
    - Either all operations of transaction properly reflected in database, or none are.
  - **I**solation:
    - Though multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions *(i.e., Intermediate transaction results must be hidden from other concurrently executed transactions).*
  - **D**urability: After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

- Enforced by the DBMS via Concurrency Control Schemes

Concept of a "Schedule"

- A "schedule" for a set of transactions has some rules for it to be valid
  - is a (possible) **sequence** that specifies the order in which instructions of concurrent transaction**s** are executed.
  - must consist of all instructions of those transactions.
  - must preserve order in which the instructions appear from each **individual** transaction.

- Note:
  - A transaction that successfully completes its execution will have a commit instruction as it's last performed statement .
    - By default, transaction is assumed to execute commit instruction as its last step.
  - A transaction that fails to successfully complete its execution will have an abort instruction as the last statement.

Sample Transactions

- Suppose system receives **two** transactions
  - Let T_1 transfer \$50 from *A* to *B*
  - Let T_2 transfer 10% of the balance from *A* to *B*

T_1
read (A)
$A := A - 50$
write (A)
read (B)
$B := B + 50$
write (B)
commit

T_2
read (A)
$temp := A * 0.1$
$A := A - temp$
write (A)
read (B)
$B := B + temp$
write (B)
commit

"Schedule" 1

- A schedule *S* is **serial** if for every transaction *T* in the schedule, all (the operations of) *T* are executed consecutively in the schedule.

- Below: a **serial** schedule in which T_1 is followed by T_2

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
	$B := B + temp$
	write (B)
	commit

"Schedule" 2

- Right: **another** valid **serial** schedule where T_2 is followed by T_1

- Note on "Schedule" involving N Transactions
  - ***n*!** possible serial schedules of *n* transactions
  - Once you allow the **interleaving** of operations, there many possible ways in which system can execute individual operations of transactions non-serially

T_1	T_2
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	

"Schedule" 3

- Let T_1 and T_2 be the transactions given previously.
  - Following schedule not serial schedule
  - but is ***equivalent*** to Schedule 1

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

T_1	T_2
read (A)	read (A)
$A := A - 50$	$temp := A * 0.1$
write (A)	$A := A - temp$
read (B)	write (A)
$B := B + 50$	read (B)
write (B)	$B := B + temp$
commit	write (B)
	commit

- In Schedules 1, 2 and 3, the sum A + B is preserved.

"Schedule" 4

- Following concurrent schedule:
  - does not preserve value of (*A* + *B*).

T_1	T_2
read (A)	
$A := A - 50$	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	$B := B + temp$
	write (B)
	commit

T_1
read (A)
$A := A - 50$
write (A)
read (B)
$B := B + 50$
write (B)
commit

T_2
read (A)
$temp := A * 0.1$
$A := A - temp$
write (A)
read (B)
$B := B + temp$
write (B)
commit

- Not all concurrent schedules are desirable

Interleave Transactions

- When multiple transactions execute concurrently in an uncontrolled or unrestricted manner, it might lead to several problems.
- Let's see exactly why issues occur in concurrent schedules
  - Suppose we have two transactions, T1 and T2:

T1:	T2:
read(X)	read(X)
X ← X +N	X ← X +M
write(X)	write(X)
read(Y)	
Y ← Y – N	
write(Y)	

Lost Update (1)

- Suppose
  - transactions *T*1 and *T*2 submitted approximately same time
  - log shows execution in following order
  - initially *X* = 100; *Y* = 50; *N* = 5 and *M* = 8
- (Aka Temporary Read/ Dirty Read Problem)
  - updated item is used by another transaction

Database	T_1	T_2
$X = 100, Y = 50$	$X =?, Y =?$	$X =?$
$X = 100, Y = 50$	read(X) $X = 100, Y =?$	$X =?$
$X = 100, Y = 50$	$X \leftarrow X + N$ $X = 105, Y =?$	$X =?$
$X = 100, Y = 50$	$X = 105, Y =?$	read(X) $X = 100$
$X = 100, Y = 50$	$X = 105, Y =?$	$X \leftarrow X + M$ $X = 108$
$X = 105, Y = 50$	write(X) $X = 105, Y =?$	$X = 108$
$X = 105, Y = 50$	read(Y) $X = 105, Y = 50$	$X = 108$
$X = 108, Y = 50$	$X = 105, Y = 50$	write(X) $X = 108$
$X = 108, Y = 50$	$Y \leftarrow Y - N$ $X = 105, Y = 45$	$X = 108$
$X = 108, Y = 45$	write(Y) $X = 105, Y = 45$	$X = 108$

Here, end of T_1 and T_2, ***X* should be 113, *Y* should be 45.** Update ← *X* + *N* has been lost!

T1:
read(X)
X ← X + N
write(X)
read(Y)
Y ← Y – N
write(Y)

T2:
read(X)
X ← X +M
write(X)

Lost Update (2)

- Suppose
  - transactions *T*1 and *T*2 submitted approximately same time
  - log shows execution in following order
  - initially *X* = 100*; Y* = 50*; N* = 5 and *M* = 8
- (Aka Temporary Read/ Dirty Read Problem)
  - one transaction updates an item and fails halfway

Database	T_1	T_2
$X = 100, Y = 50$	$X =?, Y =?$	$X =?$
$X = 100, Y = 50$	read(X) $X = 100, Y =?$	$X =?$
$X = 100, Y = 50$	$X \leftarrow X + N$ $X = 105, Y =?$	$X =?$
$X = 105, Y = 50$	write(X) $X = 105, Y =?$	$X =?$
$X = 105, Y = 50$	**FAILS**	read(X) $X = 105$
$X = 105, Y = 50$		$X \leftarrow X + M$ $X = 113$

T1:
read(X)
X ← X + N
write(X)
read(Y)
Y ← Y – N
write(Y)

T2:
read(X)
X ← X +M
write(X)

Discussion: How to Avoid Problems?

- If operations are interleaved arbitrarily, incorrect results may occur
  - Isolation can be **ensured trivially** by running transactions **serially.**

- **Easier to run only serial schedules?**
  - very poor throughput, serial schedules *considered unacceptable*
    - If transaction waits for an I/O operation to complete, we cannot switch CPU processor to another transaction, thus wasting valuable CPU processing time.
    - If some transaction *T* is quite long, the other transactions must wait for *T* to complete all its operations before starting.
  - Executing multiple transactions concurrently has significant benefits: full utilisation of **resources.**

Design Schedules in Advance?

- **Conclusion:** It is desirable to interleave the operations of transactions in an appropriate way. However, arbitrarily interleaved operations may lead to incorrect results.

- Question: If serial is not permitted, how should we/ system make sure that we don't run into these problems?
  - Further more, we are not able to design schedules in advance.

- Clarification:
  - we discussed "schedules" for the purpose of explain what is correct sequence of instruction execution.
  - By schedules we do not mean scheduling in advance!
    - Terminology reflects more of a "log" type semantic

Motivation

- **Baseline (Serial Correctness):**
  - Each transaction preserves database consistency when executed in isolation.
  - Therefore, any **serial** execution of a set of transactions preserves database consistency, regardless of execution order.

- Key intuition: We can try to identify groups/types of schedules that are always considered to be correct when concurrent transactions are executing.

- A concurrent set of n transactions must preserve consistency!
  - A (possibly concurrent) schedule *S* of *n* transactions is **serializable** if ***equivalent*** *to some serial schedule* of the same *n* transactions, if we can determine which non-serial schedules are ***equivalent*** to a serial schedule, we can allow these schedules to occur.

Observation

- Question: **How do we determine if non-serial schedules are correct?**
  - Study **notion of correctness** of concurrent executions
    - *"Every transaction is executed from beginning to end in **isolation** from the operations of other transactions, we get a correct end result on the database"*
  - Many notions forms of schedule equivalence give rise to notion of **Conflict serializability**

Simplified Context

- We ignore operations other than **read** and **write** instructions
  - We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes.
  - Our simplified schedules consist of only **read** and **write** instructions.

- *For a transaction that transfers $50 from account A to account B:*

1. **read**(*A*)
2. *A* := *A* – 50
3. **write**(*A*)
4. **read**(*B*)
5. *B* := *B* + 50
6. **write**(*B*)

We simply consider read/write only
read(*A*)
write(*A*)
read(*B*)
write(*B*)

Conflicts*

- Instructions I_i and I_j of different transactions T_i and T_j respectively, **conflict** if and only if
  - there exists some item Q accessed by both I_i and I_j **AND**
  - at least one of these instructions wrote Q

  1. I_i = **read**(Q), I_j = **read**(Q). I_i and I_j don't conflict
  2. I_i = **read**(Q), I_j = **write**(Q). They conflict
  3. I_i = **write**(Q), I_j = **read**(Q). They conflict
  4. I_i = **write**(Q), I_j = **write**(Q). They conflict

- a conflict between I_i and I_j forces a (logical) temporal order
  - i.e., changing their order **can** result in a different combined outcome.
- If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule
  - For example, read–read operations are not conflicting.

Conflict Equivalence **

- Two operations O_1 and O_2 are *conflicting* if
  - They in different transactions
  - They access same data item
  - At least one of them must be a write

- **Two schedules are *conflict equivalent*** if
  - Involve same actions of same transactions
  - Every pair of conflicting actions is ordered same way

- We define equivalence of schedules by *conflict equivalence*, which is the more commonly used definition

Conflict Serializability **

- **Two schedules are *conflict equivalent*** if
  - Involve same actions of same transactions
  - Every pair of conflicting actions is ordered same way

- Whilst preserving operation order within each transaction…
  - If a schedule *S* can be transformed into a schedule *S'* by a series of swaps of non-conflicting instructions, we say that *S* and *S'* are **conflict equivalent**.
  - Since we do not change orders of conflicting operations, we know outcome of both schedules is same

- We say schedule *S' is* **conflict serializable** if it is (conflict) equivalent to some serial schedule *S*

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

Schedule 3

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

Schedule 6

- Note: any conflict serializable schedule is also a serializable schedule

Conflict Serializability (3)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule <T_3, T_4>, or the serial schedule <T_4, T_3>.
- Note: Not all schedules are conflict serializable

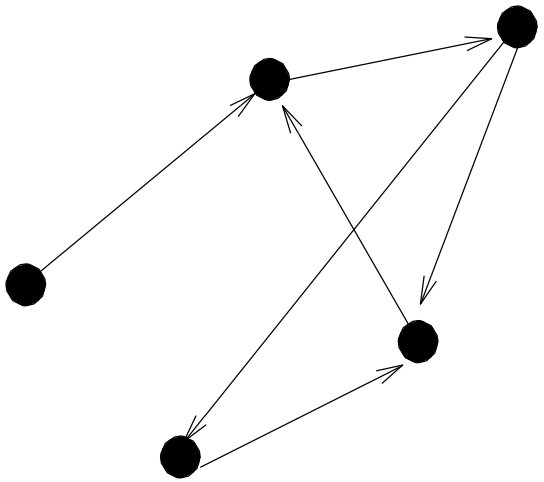
Testing Conflict Serializability

- Determining whether a schedule is conflict serializable or not?
- Simple algorithm (below)
  - Step 1: Construct a *precedence* graph.
  - Step 2: Check if the graph is *cyclic*:
    - Cyclic: non-serializable.
    - Acyclic: serializable.

- looks at only the read_item and write_item operations in a schedule
  - Recall: a Simplified View of Transactions

Preliminary Info: Directed Graphs

- A *directed graph* $G = (V, A)$ consists of
  - a vertex set V
  - an arc set A such that each arc connects two vertices.
- *Cyclic:* G is *cyclic* if G contains a directed cycle.



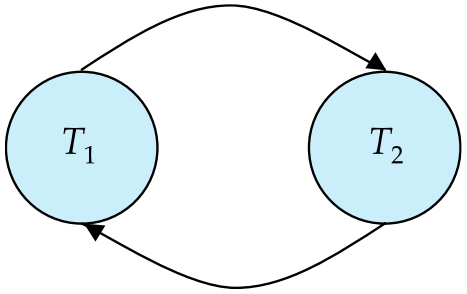
Cyclic Graph

Serializability Testing

- Consider some schedule of a set of transactions $T_1, T_2, ..., T_n$
  Precedence graph — a directed graph $G = (V, E)$ where the vertices (V) are the transactions.

- We draw an arc from T_i to T_j, $T_i \rightarrow T_j$, when we find operations in conflict, where T_i accessed the data item earlier. Such as…
  - T_i executes write(Q) before T_j executes read(Q)
  - T_i executes read(Q) before T_j executes write(Q)
  - T_i executes write(Q) before T_j executes write(Q)

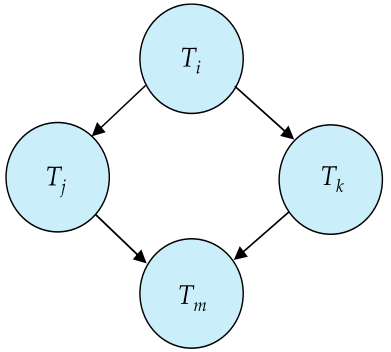
- We may label the arc by the item that was accessed.



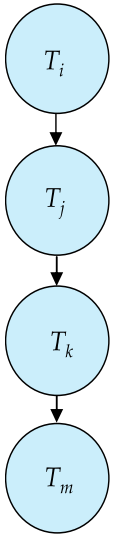
Conflict Serializability Testing

- A schedule is conflict serializable if and only if its precedence graph is acyclic (cycle free).

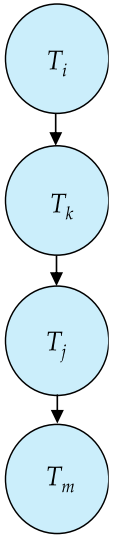
- If the precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
  - A linear order consistent with the partial order of the graph.
  - For (a), there are two linear orders (b) and (c).



(a)



(b)



(c)

Example (1)

- If there is **no** cycle in the precedence graph, this schedule is conflict-serializable
- Note: Here we label the arc by the item that was accessed.

$r_2(A); r_1(B); w_2(A); r_3(A); w_1(B); w_3(A); r_2(B); w_2(B)$

Example (2)

- If there is a cycle in the precedence graph, this schedule is NOT conflict-serializable
- Note: Here we label the arc by the item that was accessed.

$r_2(A); r_1(B); w_2(A); r_2(B); r_3(A); w_1(B); w_3(A); w_2(B)$

Concurrency Control Summary

- Key concept thus far
  - We need concurrency control, and do not tolerate serial execution
  - There are conflict serializable schedules that produces "correct" results.

- Given that…
  - Transactions are submitted to system unpredictably
  - Fixed schedules cannot be created in advance
    - We cannot not examine the precedence graph as it is being created
    - Testing a schedule for serializability like that *after* parts of the code has executed is a little too late!

- A database must provide a mechanism that will ensure that all possible schedules executed are serializable.
  - **How to control interleaving of transactions systematically to produce correct results?**

M: Tests for serializability help us understand if and why a concurrency control protocol is correct.

Lock-Based Protocols

- **Concurrency-control protocols** allow concurrent schedules, ensure that schedules are serializable.
  - imposes a discipline that avoids non-serializable schedules

- A **lock** is a mechanism to control concurrent **access to a data item**
- To **use/access** a data item, you must acquire the relevant locks, which each transaction will Transaction can only after request is granted.

- Data items can be locked in two ways, as described in the two modes:
  1. ***Exclusive Lock/ X Lock / Write Lock***
     - This data item can be read
     - This data item can be written
  2. ***Shared Lock/ S Lock/ Read Lock***
     - This data item can be read (only)

Schedule (wt. Lock-Based Protocol)

- **Lock requests** made to **concurrency-control manager**
  - An exclusive lock is requested using **write_lock()** instruction
  - A shared lock is requested using **read_lock()** instruction

- What happens next?
  - if the **requested lock** is **compatible** with locks already held on the item by other transactions, a transaction may be granted a lock on an item.
  - If **incompatible** with locks already held on the item by other transactions, it cannot be granted
    - the ***requesting transaction*** is made to wait till all locks held by other transactions have been **released**. The lock is then granted.

T_1	T_2	concurrency-control manager
write_lock(B)		
		grant-write_lock(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	read_lock(A)	
		grant-read_lock(A, T_2)
	read(A)	
	unlock(A)	
	read_lock(B)	
		grant-read_lock(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
write_lock(A)		
		grant-write_lock(A, T_1)
read(A)		
$A := A + 50$		
write(A)		
unlock(A)		

Note: Grants omitted in rest of slides. Please assume grant happens just before the next instruction following lock request

Lock Compatability Matrix

- *"Lock compatibility becomes an issue when one application holds a lock on an object and another application requests a lock on the same object."*
- Presenting the **Lock-compatibility matrix**

	S	X
S	true	false
X	false	false

- *Hint: Perhaps you now begin to notice how locking protocols enforce serializability by restricting the set of possible schedules?*

Lock-Based Protocols

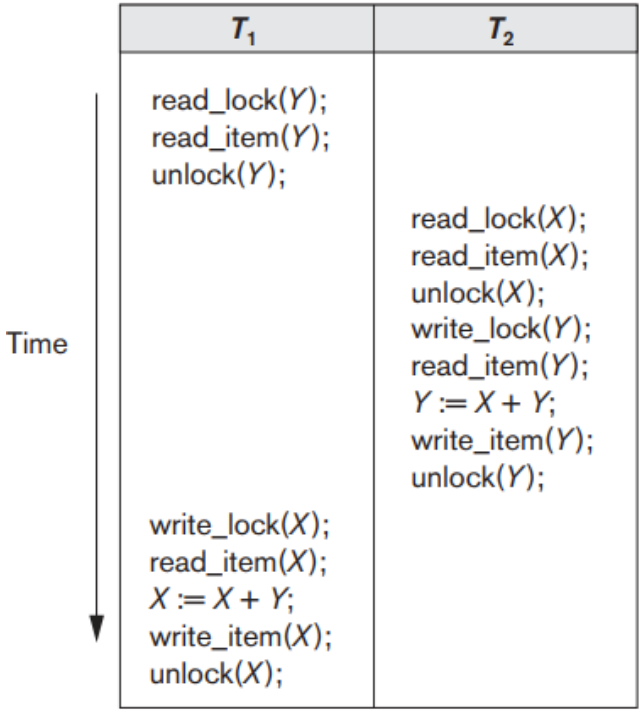
- A locking protocol is a set of rules followed by all transactions while requesting and releasing locks.

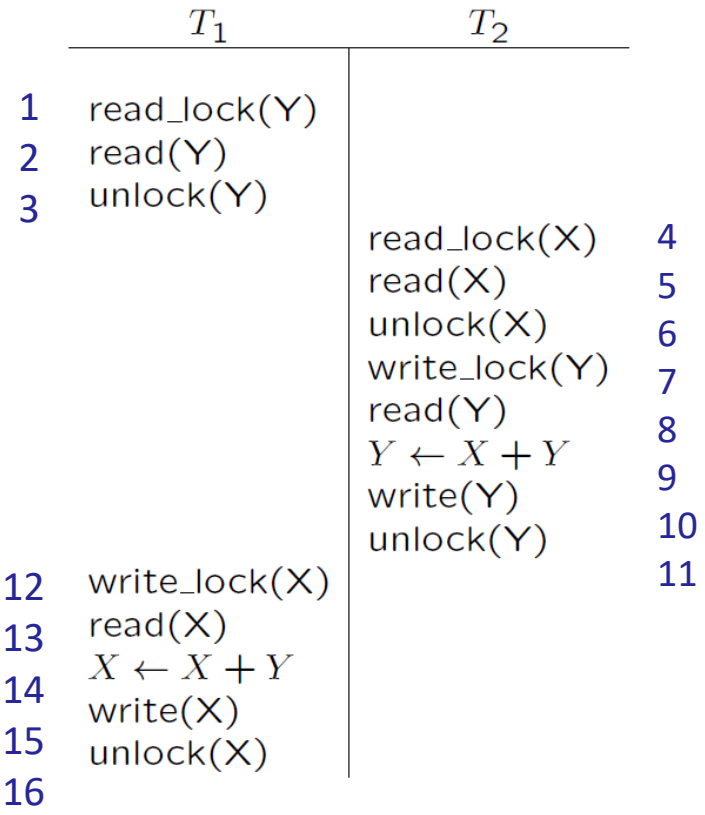
- **Locking protocol 1: "simple" locking protocol**
  - If *T* has only one operation manipulating an item *X*:
    - *if read:* obtain a read lock on *X* before reading
    - *if write:* obtain a write lock on *X* before writing
    - unlock *X after this operation on X*

- Even if *T* has many op. manipulating *X,* we obtain **one** lock still:
  - if all operations on *X* are reads, obtain read lock
  - if at least **one** operation on *X* is a write, obtain write lock
  - unlock *X* after the **last** operation on *X*

Simple Locking Protocols

- (Left hand side example with slight variation to lecture notation)





Simple Locking Protocols

- Example of transactions performing locking:

T_1	T_2
write_lock(*B*);	**read_lock**(*A*);
read(*B*);	**read**(*A*);
B: = B – 50;	**unlock**(*A*);
write(*B*);	
unlock(*B*);	**read_lock**(*B*);
	read(*B*);
write_lock(*A*);	**unlock**(*B*);
read(*A*);	**display**(*A+B*);
A: = A + 50;	
write(*A*);	
unlock(*A*);	

- Locking as above is *not sufficient* to guarantee serializability, if it unlocks the item immediately after accessing it.

Two Phase Locking (2PL)

T_3: **write_lock**(*B*);
read(*B*);
B: = B – 50;
write(*B*);
write_lock(*A*);
read(*A*);
A: = A + 50;
write(*B*);
unlock(*B*);
unlock(*A*);

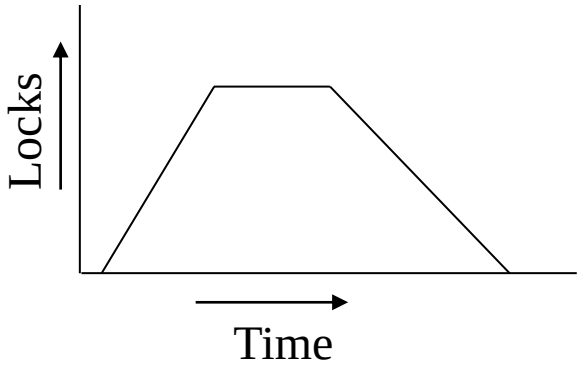
T_4: **read_lock**(*A*);
read(*A*);
read_lock(*B*);
read(*B*);
display(*A+B*);
unlock(*A*);
unlock(*B*);

- Locking as above is *sufficient* to guarantee serializability, if unlocking is delayed to the end of transaction.

Lock-Based Protocols

- **Locking Protocol Two**
  1. Phase 1: Growing Phase
     - Transaction obtains locks
     - Transaction does not release any locks
  1. Phase 2: Shrinking Phase
     - Transaction releases locks
     - Transaction does not obtain new locks
- Above "**two-phase locking protocol"** ensures serializability: and produces conflict-serializable schedules.
  - 2PL can guarantee serializability, thus allowing real-time control of concurrent executions
  - It can be proved that the transactions can be serialized in the order of their lock points (i.e., the point where a transaction acquired its final lock).

"each transaction must never acquire a lock after it has released a lock. The serializability property is guaranteed for a schedule with transactions that obey this rule"



Two Phase Locking (2PL) Example

T_1	T_2
read_lock(Y)	
read(Y)	
unlock(Y)	
	read_lock(X)
	read(X)
	unlock(X)
	write_lock(Y)
	read(Y)
	$Y \leftarrow X + Y$
	write(Y)
	unlock(Y)
write_lock(X)	
read(X)	
$X \leftarrow X + Y$	
write(X)	
unlock(X)	

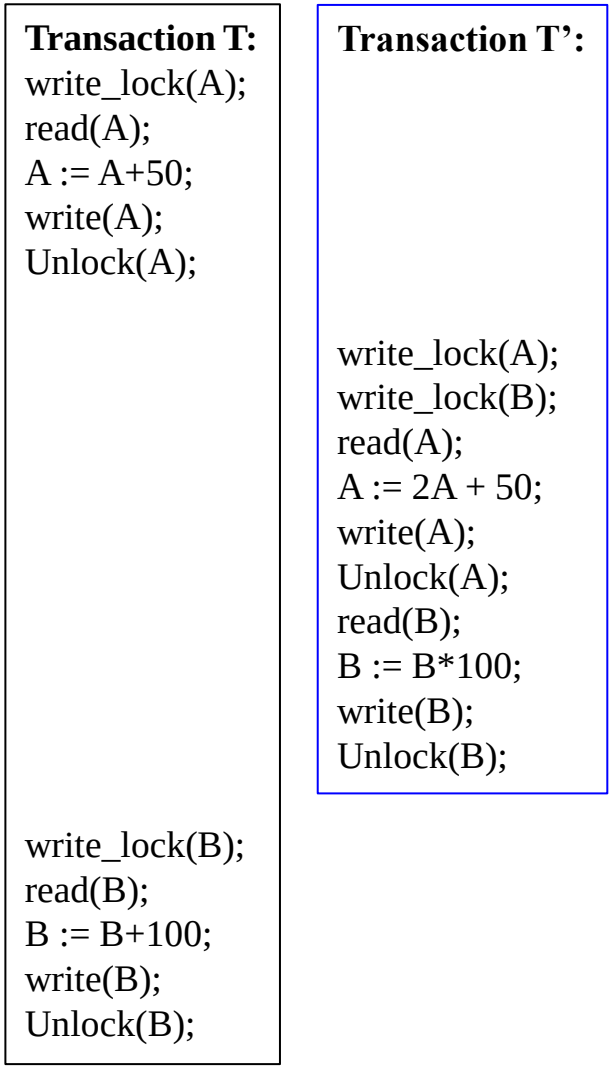
T1 that's with no the 2PL scheme

T_1
read_lock(Y)
read(Y)
write_lock(X)
unlock(Y)
read(X)
$X \leftarrow X + Y$
write(X)
unlock(X)

T1 that locks with 2PL scheme

The Two-Phase Locking Protocol (2)

- Two-phase locking is not a necessary condition for serializability. There are conflict serializable schedules that cannot be obtained if the two-phase locking protocol is used.



3 Schedule/logs with T and T'

Transaction T:	**Transaction T':**
write_lock(A);	
read(A);	
A := A+50;	
write(A);	
Unlock(A);	
	write_lock(A);
	read(A);
	A := 2A + 50;
	write(A);
	Unlock(A);
	write_lock(B);
	read(B);
	B := B*100;
	write(B);
	Unlock(B);
write_lock(B);	
read(B);	
B := B+100;	
write(B);	
Unlock(B);	

Not conflict serializable: Non two-phase looking

Transaction T:	**Transaction T':**
write_lock(A);	
read(A);	
A := A+50;	
write(A);	
Unlock(A);	
write_lock(B);	
	write_lock(A);
	read(A);
	A := 2A + 50;
	write(A);
	Unlock(A);
	write_lock(B);
read(B);	
B := B+100;	
write(B);	
Unlock(B);	
	granted the lock
	read(B);
	B := B*100;
	write(B);
	Unlock(B);

Conflict serializable: Non two-phase looking

Transaction T:	**Transaction T':**
write_lock(A);	
write_lock(B);	
read(A);	
A := A+50;	
write(A);	
Unlock(A);	
	write_lock(A);
	write_lock(B);
	read(A);
	A := 2A + 50;
	write(A);
	Unlock(A);
read(B);	
B := B+100;	
write(B);	
Unlock(B);	
	read(B);
	B := B*100;
	write(B);
	Unlock(B);

Conflict serializable: two-phase looking

Locks with 2PL

1. Use of locks, combined with the 2PL protocol, **guarantees serializability of schedules.**
2. Schedules produced by 2PL have their equivalent serial schedules based on the order in which executing transactions lock the items they acquire.
3. If transaction needs an item that is already locked, it may be forced to wait until the item is released.
4. **Some transactions may be aborted and restarted because of the deadlock problem.**

No Progress Possible

- Deadlocks occurs when *each* transaction *T* in a set of *two or more transactions* is waiting for some item that is locked by some other transaction *T* in the set.
- In most locking protocols, a deadlock can exist.
- Consider partial schedule to the right.
  - The instructions from T3 and T4 arrive at the system…
  - Such a situation is called a **deadlock**.
- executing **read_lock(B):**
  - T4 waits for T3 to release its lock on B
- executing **write_lock(A):**
  - T3 waits for T4 to release its lock on A



	T_3	T_4
1	write_lock(B)	
2	read(B)	
3	$B := B - 50$	
4	write(B)	
5		read_lock(A)
6		read(A)
7		read_lock(B)
8	write_lock(A)	

Issue: neither T3 nor T4 can make progress

Deadlock Prevention Scheme

- Locking protocols are used in most commercial DBMSs.
- **We need to address this issue of deadlocks**
- One way to prevent deadlock is to use a **deadlock prevention protocol**.

- One More Example:
  - T1: r1[x] w1[y]
  - T2: r2[y] w2[x]
  - Execution: r1[x] r2[y] deadlock!
  - T1 waits T2 to release the read lock on y, and T2 waits T1 to release the read lock on x.
  - Need a effective strategy to detect deadlocks

Deadlock Prevention Scheme (1)

- Context:
  - Lock-based concurrency control cannot prevent deadlocks.
  - We need an active solution for deadlocks beyond locks.
- **Strategy: Concurrency Control with Timeout?**
  - If a transaction waits for a period longer than a system-defined timeout period, the system assumes that the transaction may be deadlocked and aborts it.
- **Evaluation**
  - Pro: Small overhead and is simplicity.
  - Con: There may not be a deadlock

Deadlock Prevention Scheme (2)

- Context:
  - Lock-based concurrency control cannot prevent deadlocks.
  - We need an active solution for deadlocks beyond locks.
- **Strategy: Concurrency Control with Timestamps?**
  - Can assign each transaction $\mathbf{T_i}$ a "timestamp" T $\mathbf{TS(T_i)}$
  - unique identifier to identify a transaction
  - Not based on locks!
- Timestamps can be generated in several ways.
  - Possibility 1:
    - A simple **counter** (e.g., int counter)
    - (Increment value each time its value is assigned to a transaction.)
  - Possibility 2:
    - Use the current **date/time value** of the system clock.

Timestamp "Ordering"

- Concurrency control techniques based on timestamp ordering do not use locks; hence, *deadlocks cannot occur*.

- Assume T_i wants a lock that T_j holds, two policies are possible:

  - Policy 1 (**Wait-Die Protocol**):
    - If T_i is older, T_i waits for T_j
    - If T_i is younger, T_i aborts*

  - Policy 2 (**Wound-wait Protocol**):
    - If T_i is younger, T_i waits for T_j
    - If T_i is older, T_j aborts*

Note:
T_i older than T_j if **$TS(T_i) < TS(T_j)$**
* If a transaction re-starts, it retains its original timestamp

Discussion

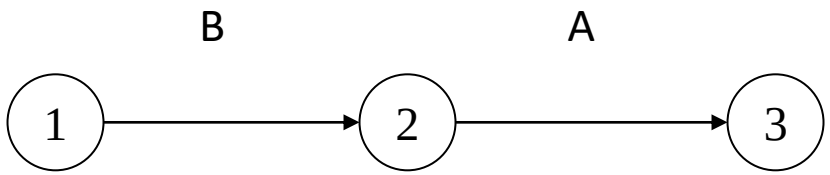
- Concurrency control via timestamp ordering
  - Ensures that result final schedule recorded is equivalent to executing transactions serially in timestamp order
  - Therefore, ensuring serializability
  - *"A different approach that guarantees serializability involves using transaction timestamps to order transaction execution for an equivalent serial schedule"*

- Compare this with concurrency control via 2PL
  - In 2PL, a schedule is serializable by being equivalent to some serial schedule allowed by the locking protocols

Solution

- If there is **no** cycle in the precedence graph, this schedule is conflict-serializable
- Note: Here we label the arc by the item that was accessed.

$r_2(A); r_1(B); w_2(A); r_3(A); w_1(B); w_3(A); r_2(B); w_2(B)$



Solution

- If there is a cycle in the precedence graph, this schedule is NOT conflict-serializable
- Note: Here we label the arc by the item that was accessed.

$r_2(A); r_1(B); w_2(A); r_2(B); r_3(A); w_1(B); w_3(A); w_2(B)$

